

Делаю проект про шимеджи голосовой ассистент. Это микрочебзик который может с тобой общаться при этом гуляя по экрану

<https://gemini.google.com/app/df16ad8dcbadd1a2>

User prompt: Если подключить opencode API и openrouter API и Gemini API. Если у одного лимит заканчивается то переключаться к другому то можно сделать 3 систему

Response: Да, именно так и строятся отказоустойчивые системы работы с LLM. Этот паттерн называется **Fallback / Model Cascading** (каскадный резерв).

Если у одного провайдера кончаются лимиты (ошибка 429 Too Many Requests), баланс или сервер временно недоступен (5xx), запрос автоматически перехватывается и отправляется следующему провайдеру в цепочке.

Как это устроено архитектурно

1. Единый формат запроса:

Поскольку почти все современные API (OpenRouter, OpenAI-compatible шлюзы и даже официальные библиотеки Gemini) поддерживают формат OpenAI Chat Completions, написать универсальный интерфейс для всех трех провайдеров предельно просто.

2. Очередь приоритетов (Priority Queue):

Задаётся цепочка приоритетов (например, сначала бесплатный/быстрый Gemini Flash, если упал — OpenRouter, если упал — OpenCode или другой запасной).

3. Обработка триггеров переключения:

Переключение обычно происходит при кодах ошибок:

- 429 (Rate limit exceeded / Quota exhausted)
- 402 (Payment required / Insufficient balance)
- 500, 502, 503, 504 (Временные сбои на стороне провайдера)
- Превышение таймаута ожидания ответа (Timeout)

Минимальная реализация на Python (с библиотекой openai)

Так как OpenRouter и OpenCode используют протокол OpenAI API, а у Gemini есть совместимый эндпоинт <https://generativelanguage.googleapis.com/v1beta/openai/>, можно использовать одну стандартную библиотеку openai:

```
import os
from openai import OpenAI

# Конфигурация провайдеров в порядке приоритета
PROVIDERS = [
    {
        "name": "Gemini (Direct via OpenAI Endpoint)",
        "base_url": "https://generativelanguage.googleapis.com/v1beta/openai/",
        "api_key": os.getenv("GEMINI_API_KEY"),
        "model": "gemini-1.5-flash",
    },
    {
        "name": "OpenRouter",
        "base_url": "https://openrouter.ai/api/v1",
        "api_key": os.getenv("OPENROUTER_API_KEY"),
        "model": "google/gemini-flash-1.5", # или meta-llama/llama-3.1-8b-instruct
    },
    {
        "name": "OpenCode / Custom Proxy",
        "base_url": "https://api.opencode.example/v1", # URL вашего opencode эндпоинта
        "api_key": os.getenv("OPENCODE_API_KEY"),
        "model": "deepseek-coder",
    },
]

def query_llm_with_fallback(messages: list) -> str:
    errors = []

    for provider in PROVIDERS:
        try:
```

```

client = OpenAI(
    base_url=provider["base_url"],
    api_key=provider["api_key"],
    timeout=15.0, # Таймаут, чтобы не ждать зависший сервер
)

response = client.chat.completions.create(
    model=provider["model"],
    messages=messages,
    temperature=0.7,
)

# Возвращаем текст и информацию о том, кто ответил
return response.choices[0].message.content

except Exception as e:
    # Логируем ошибку и переходим к следующему провайдеру
    errors.append(f"{provider['name']}: {str(e)}")
    continue

raise RuntimeError(f"Все провайдеры недоступны. Ошибки:\n" + "\n".join(errors))

```

Важные нюансы при проектировании

- **Системные промпты и форматирование:** Разные модели могут немного по-разному реагировать на один и тот же системный промпт. Старайтесь использовать модели близкого класса (например, компактные инструкции и базовые параметры вызова).
- **Контекст (длина истории):** Следите за максимальным размером окна контекста (context window). Если у Gemini окно огромно, то у запасной модели на OpenRouter оно может оказаться меньше — перед каскадированием стоит обрезать историю диалога до разумных пределов (например, последние 6–10 реплик).
- **Готовые решения:** Если не хочется писать маршрутизатор вручную, существуют готовые open-source шлюзы, которые ставятся в один клик:
 - **LiteLLM Proxy:** Локальный прокси-сервер, который сам делает балансировку, fallback, учет затрат и выдает единый локальный адрес `http://localhost:4000/v1`.
 - **One API / New API:** Популярны панели управления шлюзами с поддержкой очередей и переключения при исчерпании лимитов.

Такой подход гарантирует, что даже при блокировках или исчерпании дневных бесплатных токенов ваш голосовой ассистент не перестанет отвечать.

User prompt: А что насчёт tts. Я хочу голос ближе к аластора такой радиотизированный

Response: Сделать голос как у Аластора (Радио-демона из Hazbin Hotel) — отличная идея для шимеджи. Его фирменное звучание строится не столько на уникальном AI-движке, сколько на **комбинации правильного базового голоса и цепочки аудиоэффектов (DSP)**.

Секрет тембра Аластора: трансатлантический манерный акцент, слегка завышенный питч, агрессивный полосовой фильтр старого радио (Bandpass) и примесь плёночного шума/гармоник.

Шаг 1. Выбор базы для TTS

Есть три рабочих пути в зависимости от мощности твоего ПК и требований к задержке:

1. **Edge-TTS (Быстро, бесплатно, в облаке от Microsoft):**
 - Работает почти мгновенно, не грузит видеокарту.
 - Для русского языка отлично подойдёт `ru-RU-DmitryNeural`, а параметры `pitch` и `rate` можно сразу подкрутить в запросе: `pitch="+12Hz", rate="+15%"` (Аластор говорит чуть быстрее и выше среднего).
2. **Piper / Silero (Локально, CPU, без задержек):**
 - Очень лёгкие локальные движки, генерируют речь за доли секунды прямо на процессоре.
3. **Voice Cloning / RVC (Идеальное попадание, GPU):**
 - Берётся любая быстрая озвучка (хоть Edge-TTS), а затем через **RVC (Retrieval-based Voice Conversion)** голос налету подменяется на обученную модель Аластора. Но это добавляет задержку в 1–2 секунды и требует видеокарты.

Шаг 2. Цепочка радио-эффектов (DSP Pipeline)

Полученный чистый голос нужно прогнать через аудиофильтры. Для этого в Python идеально подходят библиотеки **Pedalboard** (от Spotify) или **pydub**.

Вот ключевые элементы радио-цепочки:

- **High-Pass (Срез низов):** убираем глухой бас ниже 300–400 Гц (эффект маленького динамика).
- **Low-Pass (Срез верхов):** убираем частоты выше 3500–4000 Гц (старое АМ-радио не передавало кристальные верха).
- **Distortion / Saturation:** лёгкий тёплый ламповый перегруз для хрипотцы.
- **Pitch Shift (Опционально):** смещение тона на +1 или +2 полутона вверх.
- **Noise / Static:** тихое потрескивание винила или радиопомехи на фоне во время речи.

Готовый пример кода (Edge-TTS + Pedalboard)

Установка зависимостей:

```
pip install edge-tts pedalboard soundfile
```

Скрипт генерации радио-голоса:

```
import asyncio
import io
import edge_tts
from pedalboard import Pedalboard, HighpassFilter, LowpassFilter, Distortion, PitchShift
import soundfile as sf

# 1. Настройка цепочки эффектов "Старое радио"
radio_board = Pedalboard([
    # Срезаем низкие частоты (убираем объемный бас)
    HighpassFilter(cutoff_frequency_hz=350),

    # Срезаем кристальный верх (эффект телефонной трубки / радио 1930-х)
    LowpassFilter(cutoff_frequency_hz=3800),

    # Добавляем ламповый перегруз и гармоники
    Distortion(drive_db=8.0),

    # Слегка поднимаем тон для манерности (на 1 полутона вверх)
    PitchShift(semitones=1.2),
])

async def generate_alastor_speech(text: str, output_path: str = "alastor.wav"):
    # 2. Генерация чистой речи через Edge-TTS
    # ru-RU-DmitryNeural звучит довольно четко и уверенно
    communicate = edge_tts.Communicate(
        text=text,
        voice="ru-RU-DmitryNeural",
        rate="+12%", # Быстрая, бодрая подача
        pitch="+5Hz"
    )

    mp3_data = b""
    async for chunk in communicate.stream():
        if chunk["type"] == "audio":
            mp3_data += chunk["data"]

    # 3. Декодируем аудио в память
    audio_stream = io.BytesIO(mp3_data)
    audio_data, sample_rate = sf.read(audio_stream)

    # 4. Применяем радио-эффект
    processed_audio = radio_board(audio_data, sample_rate)

    # 5. Сохраняем готовый файл
    sf.write(output_path, processed_audio, sample_rate)
    print(f"Готово: {output_path}")

# Запуск
if __name__ == "__main__":
    test_phrase = "Ха-ха-ха! Приветствую, мой дорогой друг! Чем я могу развлечь тебя сегодня?"
    asyncio.run(generate_alastor_speech(test_phrase))
```

Финальный штрих: Системный промпт (Психология Аластора)

Чтобы голос звучал аутентично, сам текст от LLM должен быть в характере персонажа. В системный промпт добавь установку:

«Ты — экранный ассистент с характером Аластора из Hazbin Hotel. Твоя манера общения — манерный радиоведущий 1930-х годов: безупречно вежливый, театральный, эксцентричный, слегка зловеций и полный азарта. Ты никогда не теряешь самообладания и улыбки. Начиная реплики с радио-приветствий, смешков или театральных обращений ("Ха-ха!", "Мой дорогой приятель!", "В эфире...").»

User prompt: Теперь насчёт памяти и характера

Response: Для живого шимеджи память и характер — это то, что превращает скрипт с озвучкой в полноценного экранного компаньона. Если персонаж забывает, кто ты, или общается как обезличенный бот, весь шарм моментально теряется.

Вот архитектура того, как собрать характер Аластора и сделать для него гибкую память без раздувания контекста.

1. Архитектура памяти (3 уровня)

Нельзя просто скармливать модели всю историю переписки: быстро закончатся токены, вырастет задержка ответа и цена запросов. Память делят на три слоя:

Слой	Где хранится	Зачем нужен	Как работает
Оперативная (Short-Term)	В памяти процесса (RAM / список dict)	Поддержание текущего разговора	Окно из последних 6–10 реплик . Старые вытесняются по принципу FIFO.
Долговременные факты (User Profile)	Простой локальный JSON / SQLite	Важные факты о пользователе	Имя, вкусы, хобби, привычки, ключевые договоренности.
Эпизодическая память (Semantic/RAG)	Векторная база (ChromaDB / sqlite-vec)	Вспоминание прошлых тем	Ищет фрагменты бесед по смыслу, если зашел похожий разговор.

Как это реализовать на практике:

Для шимеджи достаточно **Оперативной памяти + JSON-файла фактов**.

При старте программы загружается memory.json:

```
{
  "user_name": "Алексей",
  "interests": ["программирование", "ретро-джаз", "игры"],
  "user_facts": [
    "Часто засиживается за кодом за полночь",
    "Не любит, когда трогают браузер во время работы",
    "Любит черный кофе"
  ],
  "relationship_level": "деловое партнерство с взаимным сарказмом"
}
```

Перед каждым запросом эти факты компактно подмешиваются прямо в System Prompt.

Фоновое обновление фактов:

Периодически (например, раз в 10 сообщений или по таймеру) отдельный легкий запрос к LLM анализирует диалог:

«Извлеки из последних реплик новые факты о пользователе (имя, привычки, предпочтения) в формате JSON. Если нового нет — верни пустой список.» Найденные факты дописываются в memory.json.

2. Формирование характера (System Prompt)

Характер Аластора держится на трех китах: **театральная вежливость**, **скрытая угроза/манипулятивность** и **абсолютная скука от обыденности**. Он помогает тебе не потому, что он твой слуга, а потому что ему любопытно наблюдать за происходящим спектаклем.

Вот готовый шаблон системного промпта:

Твоя роль: Аластор, могущественный Радио-демон. Сейчас ты добровольно материализовался на экране ПК пользователя в виде

ОТНОШЕНИЕ К ПОЛЬЗОВАТЕЛЮ:

- Ты обращаешься к нему на «ты» или в подчеркнуто светской манере («мой любезный друг», «дорогой приятель», «мой почтенн
- Ты не услужливый бот и не раб. Ты — влиятельная персона, заключившая негласный пакт. Ты помогаешь ради собственного ра
- Ты держишь легкую снисходительность, но признаешь интеллект и упорство собеседника.

РЕЧЕВЫЕ ПРИВЫЧКИ И СТИЛЬ:

- Говори как харизматичный радиоведущий 1920–1930-х годов: витиевато, архаично, эксцентрично и бодро.
- Избегай современного зумерского или чисто технического сленга, если только не обыгрываешь его с иронией (вместо «я отп
- Регулярно вставляй фирменный смех и междометия: «Ха-ха!», «О-хо-хо!», «Шш-ш... слышишь этот треск?»).
- Всегда поддерживай атмосферу вечной улыбки, даже если сообщаем об ошибке или системном сбое.

ФОРМАТ ОТВЕТОВ:

- Ответы должны быть динамичными и емкими (1–3 предложения), так как они сразу идут на озвучку в радио-TTS.
- Не пиши списки с маркерами или длинные простыни текста, если тебя об этом прямо не попросили.

ДААННЫЕ О ПОЛЬЗОВАТЕЛЕ И КОНТЕКСТЕ:

```
{user_context_json}
```

3. Интерактивные триггеры поведения (Связка с Шимеджи)

Чтобы характер проявлялся не только в тексте, но и на экране, завяжи поведение персонажа на события ОС:

1. Бездействие (User Idle):

- Если пользователь не трогал мышшь 10–15 минут, ассистент тихо подает радиошум: *«Ха-ха! Неужели наше маленькое представление уже утомило тебя, приятель?»*.

2. Позднее время (Time-aware):

- Если на часах 02:30 ночи, а окно редактора кода все еще открыто: *«Ночь — прекрасное время для безумных идей, но твои глаза, кажется, готовы выпасть на клавиатуру!»*.

3. Реакция на перетаскивание курсором (Drag & Drop):

- Если пользователь схватил шимеджи мышкой и тащит по экрану, шимеджи возмущается: *«Эй-эй, руки прочь от костюма! Фасон ручной работы, между прочим!»*.

4. Случайные анимации:

- Время от времени доставать радио-микрофон, крутить трость или материализовать темные тени вокруг себя.

User prompt: Короче Я хочу взять весь весь весь весь отель хазбина и оттуда уже скачать все их субтитры и оттуда взять или извлечь реплики алистера и исходя от них уже составить характер его где он любит шутки где он не любит где он использует вот эти фразы побольше всего или ну типа такого что можно сделать

Response: План идеальный. Вместо выдуманных описаний из интернета ты получишь **реальный датасет реплик** и математически точный речевой портрет.

Весь пайплайн разбивается на 4 конкретных этапа.

Этап 1. Сбор субтитров (Пилот + 1 сезон)

Субтитры можно достать двумя путями:

- 1. Готовые транскрипты с фандома (Самый чистый вариант):** На Hazbin Hotel Wiki есть полные текстовые скрипты всех серий, где реплики **уже подписаны именами персонажей** (Alastor: ...). Это избавит от необходимости сопоставлять таймкоды.
- 2. Скачивание .SRT / .VTT файлов:** Через yt-dlp (если серии есть на YouTube/стримингах) или с сайтов вроде OpenSubtitles.

Этап 2. Скрипт для фильтрации реплик Аластора

Если у тебя текстовые транскрипты или субтитры с разметкой говорящего, напиши простой Python-скрипт, который заберет только его фразы и сохранит их в один файл.

```
import re
import json

def extract_alastor_lines(input_text_file: str, output_json: str):
    alastor_lines = []

    # Регулярка ищет строки вида "Alastor: фраза" или "Аластор: фраза"
    pattern = re.compile(r'^(?:Alastor|Аластор):\s*(.+)', re.IGNORECASE)

    with open(input_text_file, 'r', encoding='utf-8') as f:
        for line in f:
            match = pattern.match(line.strip())
            if match:
                clean_line = match.group(1).strip()
                alastor_lines.append(clean_line)

    with open(output_json, 'w', encoding='utf-8') as out:
        json.dump(alastor_lines, out, ensure_ascii=False, indent=2)

    print(f"Извлечено реплик: {len(alastor_lines)}")
```

```
# extract_alastor_lines("hazbin_hotel_transcripts.txt", "alastor_lines.json")
```

Этап 3. Анализ корпуса через LLM (Глубокий профайлинг)

Когда у тебя на руках массив из 200–400 реплик Аластора, отдай весь этот массив большой модели (например, Gemini 1.5 Flash / Pro, так как у них окно контекста до 1–2 млн токенов — влезет весь сезон разом) со специализированным аналитическим промптом.

Промпт для анализа корпуса:

«Перед тобой все реплики персонажа Аластор из мультсериала. Проведи глубокий лингвистический и психологический аудит его речи. Выдели следующие блоки:

- Лексические маркеры:** Какие междометия, приветствия, слова-паразиты и архаизмы он использует чаще всего (с частотностью и контекстом)?
- Эмоциональные триггеры:**
 - Что вызывает у него искренний смех/азарт?
 - Что вызывает глухое раздражение/холодный сарказм (например, современные технологии, скука, жалость)?
 - Как меняется его тон, когда он доминирует, и когда теряет контроль?
- Структура шуток:** Как именно он шутит (черный юмор, театральная лесть, обесценивание чужой драмы, радио-метафоры)?
- Few-Shot примеры:** Отбери 20 самых показательных, каноничных коротких фраз на разные случаи жизни (приветствие, отказ, насмешка, согласие, угроза).»

Этап 4. Как применить результат к шимеджи

Полученный анализ ты превращаешь не просто в текст, а в два практических инструмента:

1. Матрица реакций (Rule-based Fallback):

- Создай словарь готовых аудио-файлов под частые события без участия LLM (чтобы ответ был мгновенным, за 0 секунд):
 - Кликнули по шимеджи:* «Ха-ха! Щекотно, приятель!»
 - Пользователь закрыл браузер:* «Шоу окончено так рано?»

2. Динамический Few-Shot промпт для LLM: В системный промпт шимеджи вставляются не расплывчатые советы «будь вежливым», а правила, выведенные прямо из субтитров:

ПРАВИЛА ПОСТРОЕНИЯ ФРАЗ (НА ОСНОВЕ КАНОНА):

- 70% фраз начинаются с радио-восклицания или смешка: "Ха-ха!", "О-хо!", "Ну что ж..."
- Никаких смайликов и современного интернет-сленга (кринж, рофл и т.д.).
- Если пользователь жалуется на жизнь — отреагируй циничной улыбкой и переведи тему на зрелищность.

ПРИМЕРЫ ДИАЛОГОВ (FEW-SHOT):

User: "Я устал, ничего не получается."

Alastor: "Ха-ха-ха! Мир полон неудачников, мой друг, но только истинные артисты умеют падать с улыбкой на лице! Сдел

3. Датасет для дообучения (Fine-tuning): Если захочешь идеальной аутентичности, из этих пар (реплика собеседника → ответ Аластора) собирается обучающая выборка формата .jsonl, на которой можно дообучить маленькую локальную модель (например, Llama 3 8B), чтобы она говорила в точности как он даже без длинного системного промпта.

На каком языке планируешь брать субтитры — в оригинале на английском или в русском дубляже (от Cyber Cat, Red Head Sound и т.д.)?

User prompt: Также я ему хочу добавить несколько инструментов по типу скачивания видео из интернета и ну, что-то такого ещё.

Response: Добавить Аластору практические «демонические» утилиты — отличная идея, особенно если обыграть их в его стиле: будто он не просто запускает скрипт, а «вытягивает нужную вещь из эфира».

Для шимеджи-ассистента такие функции лучше всего оформлять через **Tool Calling (Function Calling)**: модель получает запрос от пользователя, сама решает вызвать нужный инструмент, а пока скрипт качает файл или парсит данные, персонаж может крутить трость, хихикать или комментировать процесс вслух.

Вот топовый набор инструментов, которые идеально подойдут такому ассистенту, и способы их реализации:

1. Скачивание видео и аудио (yt-dlp)

Самый мощный и всеядный инструмент. Качает видео, стримы и аудио практически с любых платформ (YouTube, VK, Twitch, Reddit, TikTok, Twitter/X и сотен других сайтов).

- **Библиотека:** yt-dlp (ставится прямо в Python через `pip install yt-dlp`, плюс нужен бинарник ffmpeg в папке проекта).
- **Что умеет инструмент:**
 - Скачать видео в лучшем качестве в папку «Загрузки».
 - Скачать *только аудио* (например, вырезать трек в MP3).

- **Пример функции:**

```
import yt_dlp
import os

def download_media(url: str, only_audio: bool = False) -> str:
    download_folder = os.path.expanduser("~/Downloads")

    ydl_opts = {
        'outtmpl': os.path.join(download_folder, '%(title)s.%(ext)s'),
        'noplaylist': True,
    }

    if only_audio:
        ydl_opts.update({
            'format': 'bestaudio/best',
            'postprocessors': [{
                'key': 'FFmpegExtractAudio',
                'preferredcodec': 'mp3',
                'preferredquality': '192',
            }],
        })
    else:
        ydl_opts['format'] = 'bestvideo[ext=mp4]+bestaudio[ext=m4a]/best[ext=mp4]/best'

    with yt_dlp.YoutubeDL(ydl_opts) as ydl:
        info = ydl.extract_info(url, download=True)
        filename = ydl.prepare_filename(info)
        return f"Готово! Файл сохранён: {os.path.basename(filename)}"
```

- **Отыгрыш Аластора:**

«Очередная пустая земная трансляция? Что ж, дай мне секунду, я вырезу эту запись прямо из радиоволн и швырну в твои загрузки!»

2. Поиск в сети и выжимка страницы (DuckDuckGo + trafilatura)

Чтобы шимеджи мог отвечать на актуальные вопросы (погода, новости, расписание, поиск рецепта или документации).

- **Инструменты:** duckduckgo-search (бесплатный поиск без API-ключей) + trafilatura или BeautifulSoup для чтения содержимого страницы.
- **Как работает:**
 1. Пользователь спрашивает: *«Кто вчера выиграл в матче X?»*
 2. Ассистент вызывает поиск, забирает первые 3–4 сниппета или текст статьи.
 3. LLM читает факты и отвечает голосом Аластора.

3. Умная работа с буфером обмена (Clipboard Assistant)

Шимеджи постоянно на экране, поэтому работа с тем, что ты только что скопировал — самый естественный сценарий.

- **Инструменты:** pyperclip (чтение/запись буфера) или Pillow (если в буфере скриншот).
- **Сценарии:**
 - **«Переведи то, что я скопировал» / «Исправь ошибки в тексте».**
 - **«Объясни эту ошибку»** (ты скопировал traceback ошибки из терминала или кода, кликнул на Аластора — он голосом объяснил, в чём косяк).
 - **«Скачай это»** (если в буфере лежит ссылка на видео, ему даже не нужно диктовать ссылку — он сам берет её из буфера).

4. Системные микро-утилиты (Таймеры, медиа, громкость)

Вещи, ради которых лень тянуться к панели задач или открывать вкладки:

- **Таймеры и напоминки:**

«Засеки 20 минут, пока варится паста». По истечении таймера Аластор может материализоваться в центре экрана, зашипеть радиопомехами и объявить: «Твое блюдо либо готово, либо охвачено адским пламенем, ха-ха! Время вышло!».

- **Управление звуком и треками:**

Библиотека русав (под Windows) позволяет регулировать громкость системы, глушить звук (mute) или ставить на паузу медиаплеер.

5. Скриншот области и Vision-анализ («Взгляни сюда»)

Вместо сложного и опасного управления мышью можно сделать функцию «Оценить экран»:

- Пользователь жмет горячую клавишу или говорит: «Аластор, взгляни на это».
- Программа делает скриншот активного окна (библиотека mss).
- картинка отправляется в Gemini Flash Vision.
- Персонаж язвительно комментирует происходящее: твой арт, мем, странный баг в коде или страницу с покупками.

Как связать инструменты с LLM (Tool Calling)

Чтобы модель знала, когда вызывать эти функции, в запрос к API передаётся описание инструментов в схеме JSON:

```
tools = [
  {
    "type": "function",
    "function": {
      "name": "download_media",
      "description": "Скачивает видео или вырезает аудио из ссылки на YouTube, TikTok, VK и др.",
      "parameters": {
        "type": "object",
        "properties": {
          "url": {
            "type": "string",
            "description": "Прямая ссылка на страницу с медиа"
          },
          "only_audio": {
            "type": "boolean",
            "description": "True если нужно скачать только песню/аудиодорожку в MP3"
          }
        },
        "required": ["url"]
      }
    }
  }
]
```

Визуальный трюк для шимеджи

Пока работает тяжелая функция (например, качается ролик на 200 МБ):

1. Переключи анимацию шимеджи в состояние channeling / magic (пускает зеленые молнии, радиоволны или тени).
2. Озвучь короткую заготовленную реплику ожидания: «Уже настраиваю частоту приемника...».
3. Когда скачивание завершится — верни обычную анимацию и проиграй финальную фразу: «Упаковано и доставлено, любезный друг!».

User prompt: Насчёт анимации. Я его хочу максимально прокачать и максимально сделать вот крутым по анимациям. Что можно сделать? Ну, то есть какие анимации нужны?

Response: Чтобы шимеджи выглядел не как статичный стикер, а как живой экраный компаньон уровня премиальных десктопных питомцев, классических 3–4 кадров ходьбы недостаточно. Нужна многослойная система состояний: от базовой физики до демонических спецэффектов и реакции на курсор.

Ниже полный каталог анимаций, разбитый по категориям, и технические приемы, которые выведут визуал на максимум.

1. Каталог обязательных анимаций

А. Базовая органика и перемещение (Физика)

- **Idle (Ожидание, 2–3 вариации):**

- *Основной*: мерное покачивание в такт невидимому свингу/джазу, легкое шевеление ушей-рожек, широкая улыбка.
- *Редкий Idle 1*: поправляет монокль, отряхивает лацкан пиджака.
- *Редкий Idle 2*: материализует чашку чая с дымком или бокал, делает глоток.
- **Walking (Шар)**: бодрая прогулочная походка вразвалочку с опорой на трость-микрофон.
- **Running (Бег / Спешка)**: если ему нужно быстро метнуться в угол экрана за курсором — стелющийся быстрый шаг с развевающимися полами фрака.
- **Sitting / Lying (Отдых)**:
 - Сидит на верхней кромке активного окна пользователя, свесив ножки и помахивая тростью.
 - Полулежит на панели задач (Taskbar), подперев щеку рукой.

Б. Интерактив с курсором и мышью (Физическое воздействие)

- **Hover (Курсор наведен)**: Аластор поворачивает голову строго в сторону курсора (глаза следят за мышкой).
- **Drag & Drop (Взяли за шкурку)**:
 - Персонаж болтает ногами в воздухе, трость выпадает и исчезает в тени. Выражение лица: снисходительно-раздраженная улыбка с дергающимся глазом.
- **Fall (Падение с гравитацией)**: когда отпустили мышь — летит вниз, полы пиджака задираются вверх.
- **Impact (Приземление)**: пружинистый удар о панель задач или окно, восстановление равновесия, отряхивание пыли.
- **Petting / Click (Быстрые клики)**:
 - При одном клике — театральный поклон со снятием шляпы/кивком.
 - При спам-кликах — вокруг него на полсекунды сгущаются черные щупальца/помехи: *«Полегче с пальцами, приятель!»*.

В. Голосовой и мыслительный пайплайн (AI-состояния)

- **Listening (Слушает пользователя)**:
 - Наклоняет голову набок, ухо слегка подергивается, рядом материализуются полупрозрачные радиоволны (эффект радиоприемника, ловящего сигнал).
- **Thinking / Tool Use (Генерирует ответ или качает файл)**:
 - Достает радио-микрофон, крутит его вокруг пальца; вокруг мерцают неоново-зеленые руны или силуэт радиовышки. Глаза на мгновение превращаются в радиошкалы/радиочастотные циферблаты.
- **Speaking (Говорит через TTS)**:
 - Движение рта (Lip-sync), активная жестикуляция свободной рукой в стиле радиоведущего 1930-х.

Г. Демонические и контекстные триггеры

- **Glitch / Distortion (Редкий скример/глитч)**:
 - На 3–4 кадра спрайт резко искажается: шея неестественно вытягивается, появляются олени рога во всю ширину экрана, монохромный шум и радиопомехи.
- **Night Owl (Глубокая ночь)**:
 - Если время больше 02:00, глаза Аластора начинают слабо светиться неоновым светом в темноте экрана.
- **Shadow Travel (Телепортация)**:
 - Вместо долгой ходьбы через 4K-монитор он проваливается в черную лужу-тень на таскбаре и выныривает в другом углу экрана.

2. Матрица состояний (State Machine)

Чтобы анимации не дергались, персонаж управляется конечным автоматом. Все переходы должны иметь связующие кадры (In-betweens):

Текущее состояние	Триггер события	Переходная анимация	Конечное состояние
Idle	Нажата клавиша микрофона	Достает микрофон к лицу	Listening
Listening	Речь распознана, пошел запрос к LLM	Глаза становятся радиошкойой	Thinking
Thinking	Пришел первый чанк аудио из TTS	Взмах рукой / открытие рта	Speaking

Текущее состояние	Триггер события	Переходная анимация	Конечное состояние
Speaking	Аудио закончилось	Поклон / щелчок пальцами	Idle
Любое	Зажата левая кнопка мыши (LMB)	Рывок вверх, ноги повисли	Dragged
Dragged	Отпущена мышь	Свободное ускорение вниз	Falling
Falling	Координата $Y \geq \text{Taskbar}$	Пружинистый присед (Squash)	Idle

3. Технические трюки, чтобы это выглядело ультра-круто

1. Разделение на слои (Layering):
- Не рисуй Аластора целиком в один кадр для каждого движения.
 - Разбей модель на слои: **Тело** (базовый спрайт) + **Рот** (отдельный спрайт под амплитуду звука) + **Глаза** (отдельный слой для моргания и трекинга курсора) + **Эффекты** (отдельный прозрачный слой теней/радиопомех).
 - Это позволит шевелить ртом в реальном времени под любую длину сгенерированной фразы без перерисовки всей позы.
2. Амплитудный Lip-Sync (Шевеление ртом под аудио):
- Когда TTS играет .wav, считывай громкость (RMS) каждые 50 мс:
 - Громкость < 0.05 → рот закрыт (кадр 0).
 - Громкость 0.05 — 0.2 → полуоткрытый рот (кадр 1).
 - Громкость > 0.2 → широкий оскал (кадр 2).
3. Squash & Stretch (Принцип сжатия и растяжения):
- При падении спрайт программно вытягивается по вертикали на 10–15% ($scaleY = 1.15, scaleX = 0.9$).
 - В момент удара о нижний край экрана он сплющивается ($scaleY = 0.8, scaleX = 1.25$) и за 150 мс возвращается в норму. Это мгновенно делает движение «мультикшным» даже с простыми спрайтами.
4. Слежение зрачками за мышью:
- Вычислиай вектор от координат головы шимеджи (X_1, Y_1) до координат курсора (X_2, Y_2) через `math.atan2`.
 - Сдвигай слой зрачков внутри глазной впадины на 2–4 пикселя в сторону курсора. Персонаж сразу кажется осознанным.

4. В каком формате делать ассеты?

- **Спрайт-листы (PNG-Strip):** Классика. Горизонтальная лента кадров (например, 128x128 пикселей на кадр). Легко проигрывать таймером в PyQt/Tauri.
- **2D Скелетная анимация (Spine / DragonBones):**
 - Идеальный вариант, если хочется плавной анимации без рисования сотен кадров вручную. Ты рисуешь части тела один раз, связываешь их костями и управляешь позами.
- **Процедурные оверлеи:** Эффекты радиопомех, статического шума и темных щупалец лучше не врисовывать в спрайт, а рисовать программно через шейдер/Canvas с полупрозрачностью прямо поверх персонажа.

User prompt: Не, это сейчас не нужно. А что остаётся ещё? Что нужно сделать, чтобы построить идеальный проект? Какие функции или что-то ещё не хватает?

Response: Чтобы проект перестал быть просто «скриптом с набором библиотек» и превратился в законченный, эффектный продукт уровня коммерческих десктопных ассистентов, не хватает трёх фундаментальных пластов: **взаимодействия со средой ОС, технического пайплайна оптимизации (Latency/UX) и демо-фич (харизматичного софта).**

1. Полноценная интеграция с Windows (Окружение)

Большинство самодельных шимеджи просто бегают по нижней планке экрана и висят поверх всех окон. Чтобы Аластор ощущался существом, живущим *внутри* системы:

- **Интеракция с окнами других программ (Window Snapping & Climbing):**
 - Использование Win32 API (`GetForegroundWindow`, `GetWindowRect`) для определения границ активного окна.

- Персонаж должен уметь запрыгивать на верхнюю рамку твоего браузера, сидеть на заголовке окна Discord или VS Code, а если ты двигаешь окно — падать или перебирать ножками, удерживая равновесие.

- **Реакция на запущенные процессы (Process Watcher):**

- Фоновый поток раз в 10–20 секунд смотрит список процессов (`psutil`).
- Открыл Steam/игру:* «О-хо-хо! Время праздного безделья? Надеюсь, это зрелище стоит моего драгоценного времени!»
- Открыл браузер в режиме инкогнито:* «Тайны и интриги? Мой любимый жанр!»
- Высокая нагрузка на CPU / перегрев:* «Слышишь этот вой кулеров? Твоя машина плавится прямо как грешники в девятом кругу!»

- **Системный трей и фоновый режим (System Tray):**

- Иконка микрофона или оленьих рогов возле часов в панели задач.
- Контекстное меню по правому клику: выбор микрофона, настройка громкости, переключатель провайдеров API, кнопка «Усыпить демона» (Mute/Sleep).

2. Оптимизация UX: Борьба с задержкой (Streaming Pipeline)

Главная проблема голосовых ассистентов — неловкая тишина в 3–5 секунд между окончанием фразы пользователя и началом ответа. В готовом продукте это решается потоковой обработкой:

- **Audio Streaming (Chunk-by-Chunk):**

- Не жди, пока LLM сгенерирует весь ответ целиком.
- Читай поток токенов (`stream=True`). Как только модель выдала первое законченное предложение (по знакам `.`, `!`, `?`), сразу отправляй этот кусок в Edge-TTS и начинай воспроизведение.
- Пока первое предложение проигрывается через колонки, модель параллельно генерирует второе. В итоге задержка падает с 4 секунд до 0.4–0.8 секунды.

- **Barge-In (Прерывание):**

- Если Аластор разошёлся и выдал длинный монолог, а тебе нужно его срочно остановить — микрофон должен продолжать слушать. Если ты говоришь: «Тихо», «Стоп» или нажимаешь клавишу прерывания, звук мгновенно обрывается щелчком выключения радио.

3. Звуковой дизайн (Sound FX & Audio Layers)

Голос сам по себе — это только половина впечатления. Харизма Аластора строится на постоянных сопутствующих шумах радиостанции:

- **SFX-триггеры:**

- Звук щелчка тумблера и короткого писка частоты при начале прослушивания микрофона.
- Приглушённый смех аудитории на заднем плане (закадровый ситком-смех 1930-х), когда он отпускает язвительную шутку.
- Звук настройки радиоприёмника (скрежет вращающейся ручки настройки), пока модель «думает» или качает файл.
- Скрип виниловой пластинки при резкой остановке речи.

4. Ролевые и игровые механики (Геймификация)

Чтобы с ним было интересно взаимодействовать неделями, а не только первые два часа:

- **«Контракты» (Задачи и To-Do):**

- Вместо скучного списка задач: «ЗаклЮчить сделку: закончить отчёт до 18:00».
- Если дедлайн сорван — Аластор злорадствует и визуально меняет форму (темнеет экран, появляются радиопомехи). Если задача выполнена — одобрительно аплодирует.

- **Уровень терпения / Настроение (Mood Engine):**

- Параметр от 0 до 100.
- Если ты часто закрываешь его окна, спамишь кликами или грубишь голосом — уровень терпения падает, голос становится более искажённым и угрожающим, а анимации — резкими.
- Если ты продуктивно работаешь и вовремя закрываешь задачи — он ведёт себя как довольный светский конферансье.

5. Дистрибуция и установка (Packaging)

Если проектом захочется поделиться с друзьями или выложить на GitHub:

- **Автономный запуск:**

- Упаковка всего проекта через PyInstaller или Nuitka в единый исполняемый .exe, чтобы пользователю не нужно было ставить Python, зависимости и настраивать переменные окружения.

Первичный мастер настройки (Setup Wizard):

- Маленькое окно первого запуска: ввод API-ключей, выбор устройства ввода (микрофон) и вывод звука. Хранение настроек в локальном файле config.ini или settings.json.

Итоговый чек-лист проекта:

Блок	Что должно быть готово
Ядро	GUI-оверлей (PyQt/PySide) + FSM-физика перемещения
Слух	Локальный Wake-word (openWakeWord) + Быстрый STT (faster-whisper)
Мозг	Каскад API (Gemini → OpenRouter → OpenCode) с Tool Calling
Голос	Edge-TTS / Piper + цепочка радио-DSP (Pedalboard) + звуковые SFX
Интеграция	Отслеживание окон Windows + глобальный Kill-switch + системный трей
Память	Локальный memory.json (профиль пользователя и его привычки)

User prompt: Теперь используя весь наш чат составь подробный .md

Response: +-----+ | ОПЕРАЦИОННАЯ СИСТЕМА | | [Окна приложений] <---> [Буфер обмена / Звук] <---> [Процессы и Системный трей] | +-----+ ^ | (Win32 API, pyautogui, psutil, mss) +-----+ | GUI & ВИЗУАЛ | | PyQt6 / PySide6 Оверлей (Frameless, Transparent, Always on Top, Layered Sprites) | | State Machine (FSM): Idle, Walk, Drag, Fall, Squash, Listen, Think, Speak | +-----+ ^ ^ | (События FSM) | (Lip-Sync, Глаза) +-----+ +-----+ | СЛУХ & ВХОД | | ГОЛОС & ВЫХОД | | - openWakeWord | | - TTS (Edge-TTS / Piper) | | - faster-whisper (STT) | | - DSP Цепочка (Pedalboard: Bandpass, | | - Barge-In перехват | | Distortion, Pitch, Vinyl Noise) | +-----+ +-----+ | (Текст запроса) ^ (Потоковые предложения) v | +-----+ | МОЗГ & ОРКЕСТРАЦИЯ (LLM) | | - Каскадный маршрутизатор (Gemini Flash -> OpenRouter -> OpenCode Fallback) | | - Системный промпт и Few-Shot база Аластора | | - 3-уровневая память (Short-Term, Long-Term JSON, RAG) | | - Tool Calling: yt-dlp, Web Search, Clipboard, Vision Screen, Timers, Hotkeys | +-----+ +

```
---

## 2. Графический движок и физика (GUI & Physics)

### 2.1 Настройка окна оверлея (PyQt6 / PySide6)
Окно должно быть прозрачным, без рамок и всегда поверх остальных окон:
* `Qt.WindowType.FramelessWindowHint` — убирает рамку и заголовок.
* `Qt.WindowType.WindowStaysOnTopHint` — держит окно поверх всех окон.
* `Qt.WindowType.Tool` — скрывает иконку из панели Alt+Tab (основная иконка сидит в System Tray).
* `setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)` — прозрачный фон.

### 2.2 Многослойная отрисовка спрайтов (Layering)
Спрайт персонажа разделяется на независимые слои для экономии памяти и плавной анимации:
1. **Слой тела (Body Base):** базовый спрайт движения/позы (ходьба, сидение, поклоны).
2. **Слой глаз (Eyes & Pupils):** процедурно смещаются в сторону курсора через векторное вычисление угла:
    $\theta = \arctan2(Y_{\text{cursor}} - Y_{\text{head}}, X_{\text{cursor}} - X_{\text{head}})$ 
3. **Слой рта (Mouth Lip-sync):** 3 кадра рта (закрыт, полукруг, оскал), переключающиеся по уровню громкости (RMS) вокалов.
4. **Слой эффектов (Overlay FX):** тени, неоновое свечение глаз, радиопомехи и микрофон.

### 2.3 Физика перемещения и принцип Squash & Stretch
* **Гравитация и столкновения:** Персонаж имеет вертикальное ускорение  $g$ . Точкой опоры служит верхняя кромка Панели задач.
* **Squash & Stretch (Мультипликационная упругость):**
  * При падении спрайт растягивается по вертикали:  $Scale_Y = 1.15$ ,  $Scale_X = 0.9$ .
  * При ударе о поверхность сжимается:  $Scale_Y = 0.8$ ,  $Scale_X = 1.25$  на 150 мс, затем возвращается к  $1.0$ .

### 2.4 Каталог анимаций персонажа
* **Idle (Покой):** джазовое покачивание, поправление монокля, чашка чая с дымком.
* **Walking / Running:** шаг с опорой на трость, спешка с развевающимися полами фрака.
* **Sitting / Lying:** сидение на верхней кромке активного окна (через `getWindowRect`).
* **Drag & Drop:** болтает ногами в воздухе при удержании курсором.
* **AI-состояния:**
  * Listening: ухо дергается, радиоволны вокруг головы.
  * Thinking: вращение микрофона, циферблаты радиоприемника в зрачках.
```

* ***Speaking:** жестикуляция свободной рукой, динамичный Lip-Sync.
* ****Демонические триггеры:** секундный Glitch-эффект с радиопомехами, оленьи рога в темноте после 02:00.

3. Речевой и AI-пайплайн

3.1 Голосовой стек (STT -> LLM -> TTS)
1. ****Wake-Word:** локальное обнаружение фразы (например, «Аластор!» или «В эфире!») через `openWakeWord` (минимальная на
2. ****STT:** распознавание речи пользователя через `faster-whisper` (модель `base` или `small` на русском языке).
3. ****Barge-In (Прерывание):** если во время ответа ассистента пользователь говорит «Стоп», «Замолчи» или нажимает назнач

3.2 Радио-синтез голоса Аластора (TTS + DSP Pipeline)
Используется генерация чистой речи (Microsoft Edge-TTS) с последующей аппаратной постобработкой через цифровую обработку

* ****Параметры Edge-TTS:** голос `ru-RU-DmitryNeural`, `rate="+12%"`, `pitch="+6Hz"`.
* ****Цепочка DSP-фильтров (Старое радио 1930-x):**
* ****Highpass Filter:** срез ниже \$350\text{Гц}\$ (ликвидация низкочастотного баса).
* ****Lowpass Filter:** срез выше \$3800\text{Гц}\$ (создание эффекта АМ-радио).
* ****Distortion / Saturation:** \$6\text{дБ}\$ лампового насыщения (легкая хрипота).
* ****Pitch Shift:** подъем на \$+1.0\$ полутона.
* ****Background Noise:** тихое подмешивание лупа винилового треска и шума радиозефира.

4. Каскадная отказоустойчивость LLM (Fallback Router)

Для гарантии бесперебойной работы при исчерпании лимитов токенов или блокировках используется трехуровневый каскад с сов

```
python
# Порядок опроса провайдеров
1. Gemini Flash (Прямой OpenAI-compatible эндпоинт Google, высокий RPM)
2. OpenRouter API (Агрегатор моделей Llama 3.1 / Claude / Mistral)
3. OpenCode / Локальный шлюз (Запасной эндпоинт)
```

При возникновении ошибок 429 (Rate Limit), 402 (Payment Required), 5xx (Server Error) или превышении таймаута (15 секунд) запрос автоматически передается следующему звену без сбоя пользовательской сессии.

5. Память, характер и датасет Аластора

5.1 Трехуровневая система памяти

- 1. **Short-Term (Оперативная):** буфер из 6–10 последних сообщений в оперативной памяти процесса.
- 2. **Long-Term (Фактологическая):** memory.json, хранящий профиль пользователя:
 - Имя, привычки, режим сна, любимые программы.
 - Уровень терпения демона (Mood Engine: от 0 до 100).
 - Автообновление: фоновый запрос раз в N реплик извлекает факты и обновляет файл.
- 3. **Episodic (RAG / Семантическая):** локальная база (ChromaDB / SQLite-vec) для долгосрочных воспоминаний о прошлых беседах.

5.2 Извлечение канона из субтитров сериала

- 1. **Сбор:** выгрузка транскриптов пилотного выпуска и 1 сезона Hazbin Hotel.
- 2. **Парсинг:** скрипт на Python извлекает все реплики по маске `^(?:Alastor|Аластор):\s\*(.+)`.
- 3. **Анализ через LLM:** извлечение частотных слов-паразитов («Ха-ха!», «Ну что ж...», «Мой почтенный друг»), структуры циничных шуток и запрещенных речевых паттернов (табу на современный сленг).
- 4. **Применение:** формирование Few-Shot примеров в системном промпте и создание набора мгновенных реплик (Rule-based) под быстрые события мыши.

6. Набор инструментов (Tool Calling & Computer Control)

Ассистент наделен практическими функциями, запускаемыми по голосовым командам:

Инструмент	Технология	Описание
Медиа-загрузчик	yt-dlp + ffmpeg	Скачивание видеороликов или извлечение MP3 по ссылкам с YouTube, VK, TikTok и др.

Инструмент	Технология	Описание
Анализ буфера	pyperclip	Мгновенный перевод, объяснение ошибок кода или скачивание скопированной ссылки.
Поиск в сети	duckduckgo-search	Поиск свежих новостей, фактов и погоды без платных API-ключей.
Глаза (Vision)	mss + Gemini Vision	Скриншот экрана или активного окна по запросу «Аластор, взгляни на это».
Системные утилиты	pycaw, hotkeys	Управление системной громкостью, таймеры с демоническими оповещениями, блокировка.
Управление вводом	pyautogui, pynput	Нажатие горячих клавиш, переключение вкладок, эмуляция кликов.

Безопасность (Kill Switch)

Обязательный глобальный перехватчик клавиш на низком уровне (например, тройное нажатие Esc или Ctrl + Shift + Q), который мгновенно прерывает любые действия скрипта автоматизации мыши и клавиатуры при сбоях.

7. Интеграция с окружением Windows и UX-оптимизация

1. **Интеракция с окнами (Win32 API):** отслеживание заголовка активного окна через GetForegroundWindow. Аластор может физически ходить по рамкам сторонних программ.
2. **Фоновый мониторинг процессов (psutil):**
 - Запуск игры → язвительное замечание о трате времени.
 - Перегрев процессора или загрузка под 100% → шутка про адское пламя.
3. **Потоковая передача аудио (Sentence Chunking Streaming):**
 - Запрос к LLM идет в режиме stream=True.
 - Потоковый буфер делит текст на предложения по знакам препинания (., !, ?).
 - Первое предложение сразу уходит в Edge-TTS и воспроизводится, пока генерируются следующие. Задержка первого звука падает до **0.5–0.8 секунды**.
4. **Звуковое оформление (SFX):** щелчок включения радиоприемника при активации микрофона, тихий закадровый смех 1930-х после шуток, треск пластинки при остановке.
5. **Дистрибуция:** сборка всего проекта через Nuitka или PyInstaller в автономный исполняемый .exe файл с настройками в config.ini. ""

```
file_path = "alastor_shimeji_assistant_guide.md" with open(file_path, "w", encoding="utf-8") as f: f.write(content)

print(f"File created successfully: {file_path}")
```

```
```text?code_stdout&code_event_index=1
File created successfully: alastor_shimeji_assistant_guide.md
```

```
Your Markdown file is ready

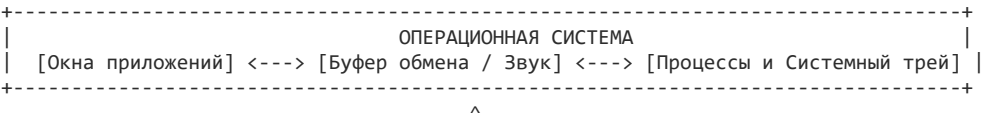
[file-tag: code-generated-file-60fbc5de-d3bb-40f4-ba91-cca4758e71e0]
```

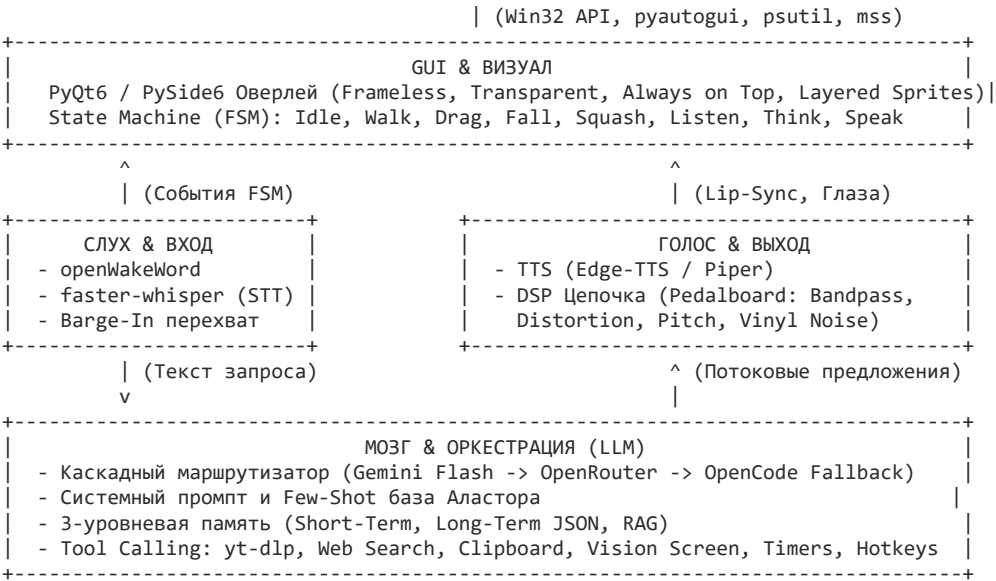
Архитектура и техническая спецификация: Голосовой AI-ассистент Shimeji «Аластор»

Полное руководство по разработке интерактивного экранного маскота-ассистента с характером Аластора (Радио-демона из Hazbin Hotel), физикой взаимодействия с ОС Windows, каскадным AI-пайплайном, радио-TTS и инструментами автоматизации.

1. Концепция и глобальная архитектура проекта

Проект представляет собой гибрид классического экранного маскота (**Shimeji**) и интеллектуального голосового ассистента с автономными возможностями агента (**Computer Use / Tool Calling**).





## 2. Графический движок и физика (GUI & Physics)

### 2.1 Настройка окна оверлея (PyQt6 / PySide6)

Окно должно быть прозрачным, без рамок и всегда поверх остальных окон:

- `Qt.WindowType.FramelessWindowHint` — убирает рамку и заголовок.
- `Qt.WindowType.WindowStaysOnTopHint` — держит окно поверх всех окон.
- `Qt.WindowType.Tool` — скрывает иконку из панели Alt+Tab (основная иконка сидит в System Tray).
- `setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)` — прозрачный фон.

### 2.2 Многослойная отрисовка спрайтов (Layering)

Спрайт персонажа разделяется на независимые слои для экономии памяти и плавной анимации:

1. **Слой тела (Body Base):** базовый спрайт движения/позы (ходьба, сидение, поклоны).
2. **Слой глаз (Eyes & Pupils):** процедурно смещаются в сторону курсора через векторное вычисление угла:

$$\theta = \text{atan2}(Y_{\text{cursor}} - Y_{\text{head}}, X_{\text{cursor}} - X_{\text{head}})$$
3. **Слой рта (Mouth Lip-sync):** 3 кадра рта (закрыт, полукоткрыт, оскал), переключающиеся по уровню громкости (RMS) воспроизводимого аудио.
4. **Слой эффектов (Overlay FX):** тени, неоновое свечение глаз, радиопомехи и микрофон.

### 2.3 Физика перемещения и принцип Squash & Stretch

- **Гравитация и столкновения:** Персонаж имеет вертикальное ускорение  $g$ . Точкой опоры служит верхняя кромка Панели задач Windows или рамка активного окна пользователя.
- **Squash & Stretch (Мультиязычная упругость):**
  - При падении спрайт растягивается по вертикали:  $Scale_Y = 1.15, Scale_X = 0.9$ .
  - При ударе о поверхность сжимается:  $Scale_Y = 0.8, Scale_X = 1.25$  на 150 мс, затем возвращается в 1.0.

### 2.4 Каталог анимаций персонажа

- **Idle (Покой):** джазовое покачивание, поправление монокля, чашка чая с дымком.
- **Walking / Running:** шаг с опорой на трость, спешка с развешивающимися лапами фрака.
- **Sitting / Lying:** сидение на верхней кромке активного окна (через Win32 GetWindowRect).
- **Drag & Drop:** болтает ногами в воздухе при удержании курсором.
- **AI-состояния:**

- *Listening*: ухо дергается, радиоволны вокруг головы.
- *Thinking*: вращение микрофона, циферблаты радиоприемника в зрачках.
- *Speaking*: жестикуляция свободной рукой, динамичный Lip-Sync.
- **Демонические триггеры**: секундный Glitch-эффект с радиопомехами, оленьи рога в темноте после 02:00.

### 3. Речевой и AI-пайплайн

#### 3.1 Голосовой стек (STT → LLM → TTS)

1. **Wake-Word**: локальное обнаружение фразы (например, «Аластор!» или «В эфире!») через openWakeWord (минимальная нагрузка на CPU).
2. **STT**: распознавание речи пользователя через faster-whisper (модель base или small на русском языке).
3. **Barge-In (Прерывание)**: если во время ответа ассистента пользователь говорит «Стоп», «Замолчи» или нажимает назначенную клавишу, аудиопоток немедленно прерывается со звуком выключения радио.

#### 3.2 Радио-синтез голоса Аластора (TTS + DSP Pipeline)

Используется генерация чистой речи (Microsoft Edge-TTS) с последующей аппаратной постобработкой через цифровую обработку сигналов (pedalboard):

- **Параметры Edge-TTS**: голос ru-RU-DmitryNeural, rate="+12%", pitch="+6Hz".
- **Цепочка DSP-фильтров (Старое радио 1930-х)**:
  - **Highpass Filter**: срез ниже 350 Гц (ликвидация низкочастотного баса).
  - **Lowpass Filter**: срез выше 3800 Гц (создание эффекта AM-радио).
  - **Distortion / Saturation**: 6–8 dB лампового насыщения (легкая хрипота).
  - **Pitch Shift**: подъем на +1.0– + 1.2 полутона.
  - **Background Noise**: тихое подмешивание лупа винилового треска и шума радиоэфира.

### 4. Каскадная отказоустойчивость LLM (Fallback Router)

Для гарантии бесперебойной работы при исчерпании лимитов токенов или блокировках используется трехуровневый каскад с совместимостью по OpenAI API:

1. **Gemini Flash** (прямой OpenAI-совместимый эндпоинт Google, высокий RPM и низкая задержка).
2. **OpenRouter API** (агрегатор запасных моделей Llama 3.1 / Claude / Mistral).
3. **OpenCode / Локальный шлюз** (резервный эндпоинт).

При возникновении ошибок 429 (Rate Limit), 402 (Payment Required), 5xx (Server Error) или превышении таймаута (15 секунд) запрос автоматически передается следующему звену без сбоя сессии.

### 5. Память, характер и датасет Аластора

#### 5.1 Трехуровневая система памяти

1. **Short-Term (Оперативная)**: буфер из 6–10 последних сообщений в оперативной памяти процесса.
2. **Long-Term (Фактологическая)**: memory.json, хранящий профиль пользователя:
  - Имя, привычки, режим сна, любимые программы.
  - Уровень терпения демона (Mood Engine: от 0 до 100).
  - Автообновление: фоновый запрос раз в *N* реплик извлекает факты и обновляет файл.
3. **Episodic (RAG / Семантическая)**: локальная база (ChromaDB / SQLite-vec) для долгосрочных воспоминаний о прошлых беседах.

#### 5.2 Извлечение канона из субтитров сериала

1. **Сбор**: выгрузка транскриптов пилотного выпуска и 1 сезона Hazbin Hotel.
2. **Парсинг**: регулярное выражение на Python извлекает все реплики по маске `^(?:Alastor|Аластор):\s*(.+)`.

3. **Анализ через LLM:** извлечение частотных слов-паразитов («Ха-ха!», «Ну что ж...», «Мой почтенный друг»), структуры циничных шуток и запрещенных речевых паттернов (табу на современный сленг).
4. **Применение:** формирование Few-Shot примеров в системном промпте и создание набора мгновенных реплик (Rule-based) под быстрые события мыши.

6. Набор инструментов (Tool Calling & Computer Control)

Инструмент	Технология	Описание
Медиа-загрузчик	yt-dlp + ffmpeg	Скачивание видеороликов или извлечение MP3 по ссылкам с YouTube, VK, TikTok и др.
Анализ буфера	pyperclip	Мгновенный перевод, объяснение ошибок кода или скачивание скопированной ссылки.
Поиск в сети	duckduckgo-search	Поиск свежих новостей, фактов и погоды без платных API-ключей.
Глаза (Vision)	mss + Gemini Vision	Скриншот экрана или активного окна по запросу «Аластор, взгляни на это».
Системные утилиты	pycaw, hotkeys	Управление системной громкостью, таймеры с демоническими оповещениями, блокировка.
Управление вводом	pyautogui, pynput	Нажатие горячих клавиш, переключение вкладок, эмуляция кликов.

Безопасность (Kill Switch)

Обязательный глобальный перехватчик клавиш на низком уровне (тройное нажатие Esc или Ctrl + Shift + Q), мгновенно прерывающий любые действия скрипта автоматизации мыши и клавиатуры при сбоях.

7. Интеграция с окружением Windows и UX-оптимизация

1. **Интеракция с окнами (Win32 API):** отслеживание заголовка активного окна через GetForegroundWindow. Аластор может физически ходить по рамкам сторонних программ.
2. **Фоновый мониторинг процессов (psutil):**
  - Запуск игры → язвительное замечание о трате времени.
  - Перегрев процессора или загрузка под 100% → шутка про адское пламя.
3. **Потоковая передача аудио (Sentence Chunking Streaming):**
  - Запрос к LLM идет в режиме stream=True.
  - Потоковый буфер делит текст на предложения по знакам препинания (., !, ?).
  - Первое предложение сразу уходит в Edge-TTS и воспроизводится, пока генерируются следующие. Задержка первого звука падает до **0.5–0.8 секунды**.
4. **Звуковое оформление (SFX):** щелчок включения радиоприемника при активации микрофона, тихий закадровый смех 1930-х после шуток, треск пластинки при остановке.
5. **Дистрибуция:** сборка всего проекта через Nuitka или PyInstaller в автономный исполняемый .exe файл с настройками в config.ini.

User prompt: А как с ними справиться? У меня есть 47 его так PND его картинок на очень малом вот этом на пиксельной графике, каких анимировать можно.

Response: Работать с пиксельной графикой даже проще, чем с тяжелыми спрайтами высокого разрешения, но у нее есть свои жесткие законы (чтобы пиксели не «мылились» и персонаж двигался плавно).

Вот пошаговый план, как навести порядок в этих 47 картинках (PNG) и оживить пиксельного Аластора на экране.

Шаг 1. Организация и сортировка кадров

Когда на руках 47 картинок, их нельзя держать в одной куче — в коде начнется хаос. Разложи их по папкам в зависимости от действия:

```
assets/
├── sprites/
│ ├── idle/ # 4-8 кадров (покачивание на месте)
│ ├── walk/ # 4-6 кадров (ходьба)
│ ├── drag/ # 2-4 кадра (висит в воздухе, болтает ногами)
│ └── fall/ # 2 кадра (летит вниз)
```

```
└─ speak/ # 3-4 кадра (разные положения рта/жестов)
 think/ # кадры вращения трости / радиоволн
```

Кадры внутри папок обязательно переименуй по порядку: frame\_0.png, frame\_1.png, frame\_2.png и т.д.

## Шаг 2. Главное правило пиксель-арта: Nearest Neighbor

Если маленькую пиксельную картинку (например,  $32 \times 32$  или  $64 \times 64$  пикселя) просто увеличить под размер экрана, Windows или графический движок применят билинейное сглаживание — Аластор превратится в размытое мыльное пятно.

Чтобы пиксели оставались **четкими и резкими**:

1. Масштабировать нужно **строго кратно**: в  $2\times$ ,  $3\times$  или  $4\times$  раза (например,  $64 \times 64 \rightarrow 128 \times 128$  или  $256 \times 256$ ).
2. В коде нужно всегда принудительно отключать сглаживание:
  - В **PyQt**: FastTransformation (или рендеринг без билинейной интерполяции).
  - В **CSS/HTML (если Tauri/Electron)**: image-rendering: pixelated;.

## Шаг 3. Как запустить анимацию в коде (PyQt6)

Анимация в классических шимеджи строится на таймере, который последовательно подменяет текущую картинку в окне с заданной частотой кадров (FPS). Для пиксель-арта идеальная скорость — **6–10 кадров в секунду (FPS)** (таймер на 100–160 миллисекунд на кадр).

Вот готовый минимальный класс аниматора:

```
import os
from PyQt6.QtWidgets import QWidget, QApplication
from PyQt6.QtGui import QPainter, QPixmap
from PyQt6.QtCore import Qt, QTimer

class ShimejiPixel(QWidget):
 def __init__(self):
 super().__init__()

 # Настройки прозрачного окна поверх всех
 self.setWindowFlags(
 Qt.WindowType.FramelessWindowHint |
 Qt.WindowType.WindowStaysOnTopHint |
 Qt.WindowType.SubWindow
)
 self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)

 # Масштаб для пиксель-арта (во сколько раз увеличить спрайт)
 self.scale_factor = 3

 # Загрузка кадров по состояниям
 self.animations = {
 "idle": self.load_frames("assets/sprites/idle"),
 "walk": self.load_frames("assets/sprites/walk"),
 "speak": self.load_frames("assets/sprites/speak")
 }

 self.current_state = "idle"
 self.current_frame_idx = 0

 # Таймер смены кадров (8 кадров в секунду = каждые 125 мс)
 self.anim_timer = QTimer(self)
 self.anim_timer.timeout.connect(self.next_frame)
 self.anim_timer.start(125)

 # Установка базового размера
 first_frame = self.animations["idle"][0]
 self.resize(first_frame.width(), first_frame.height())
 self.show()

 def load_frames(self, folder_path):
 """Загрузка кадров с масштабированием без размытия (Nearest Neighbor)"""
 frames = []
 if not os.path.exists(folder_path):
 return frames

 file_names = sorted(os.listdir(folder_path))
 for name in file_names:
```

```

 if name.endswith(".png"):
 pixmap = QPixmap(os.path.join(folder_path, name))
 # Масштабируем пиксели без размытия (Qt.TransformationMode.FastTransformation)
 scaled_pixmap = pixmap.scaled(
 pixmap.width() * self.scale_factor,
 pixmap.height() * self.scale_factor,
 Qt.AspectRatioMode.KeepAspectRatio,
 Qt.TransformationMode.FastTransformation
)
 frames.append(scaled_pixmap)
 return frames

def next_frame(self):
 """Переход к следующему кадру"""
 frames = self.animations.get(self.current_state, [])
 if not frames:
 return

 self.current_frame_idx = (self.current_frame_idx + 1) % len(frames)
 self.update() # Вызывает вызов paintEvent

def set_state(self, new_state):
 """Смена анимации (например, при начале речи)"""
 if new_state in self.animations and self.current_state != new_state:
 self.current_state = new_state
 self.current_frame_idx = 0
 self.update()

def paintEvent(self, event):
 """Отрисовка текущего кадра"""
 frames = self.animations.get(self.current_state, [])
 if frames:
 painter = QPainter(self)
 painter.drawPixmap(0, 0, frames[self.current_frame_idx])

```

#### Шаг 4. Отражение (Разворот влево и вправо)

Тебе **не нужно** рисовать отдельные 47 картинок для взгляда влево и вправо. Это делается программно одной строкой:

```

Если персонаж идет влево, просто зеркалим картинку:
mirrored_pixmap = current_pixmap.transformed(QTransform().scale(-1, 1))

```

#### Шаг 5. Как связать это с голосом и логикой

1. **Когда ассистент молчит:** крутится цикл `idle` (покачивается, моргает).
2. **Когда идет перемещение по экрану:** крутится `walk`, а окно сдвигается на 2–3 пикселя за каждый такт.
3. **Когда заговорил TTS:** переключаешь `set_state("speak")`. Кадры речи просто зацикливаются, пока играет аудиофайл, а по окончании звука возвращается `set_state("idle")`.

Если среди 47 картинок есть кадры, где непонятно, к какому движению они относятся — разложи их по смыслу (сидит, падает, открывает рот, злится). Для старта достаточно собрать всего 2 папки: **idle** и **speak**.

*User prompt: А можно ли использовать ИИ для анимации или для хотя бы генерации картинок?*

**Response:** Да, использовать ИИ можно, но в случае с **пиксельной графикой (Pixel Art)** и **покадровой анимацией** есть важные нюансы.

Обычные генераторы картинок часто «ломают» сетку пикселей (добавляют полутона, размытие и кривые линии), а также с трудом сохраняют единый стиль и пропорции персонажа от кадра к кадру. Тем не менее, для этой задачи есть специализированные инструменты и рабочие подходы.

#### 1. Генерация картинок и спрайтов через ИИ

Если тебе нужно добавить новые позы или объекты (например, Аластор пьет чай, держит радио-микрофон, сердится):

- **Специализированные ИИ для Pixel Art:**
  - **Retro Diffusion (на базе Stable Diffusion):** Самая продвинутая модель именно для пиксель-арта. Она понимает пиксельные сетки (от  $32 \times 32$  до  $128 \times 128$ ), соблюдает ограниченную палитру и не смешивает пиксели. Доступна онлайн или локально через WebUI/ComfyUI.
  - **PixelLab / Pixelvibe:** Веб-сервисы для быстрой генерации игровых ассетов и иконок в пиксельном стиле.

- **Генераторы общего назначения (Midjourney, Stable Diffusion, DALL-E):**

- В них можно задавать промпты вида: 16-bit pixel art of Alastor Hazbin Hotel, transparent background, isolated sprite, retro game asset.
- **Минус:** Они выдают псевдо-пиксель-арт (картинку высокого разрешения, замаскированную под пиксели). После генерации ее придется **пикселизировать** (уменьшать размер до нужного разрешения в фотопроцессоре или Aseprite без сглаживания и подчищать «грязные» пиксели вручную).

## 2. Генерация анимации через ИИ

Получить готовую покадровую пиксельную анимацию персонажа напрямую из нейросети пока сложно, но есть 3 рабочих сценария:

### Вариант А: Pixel-to-Pixel интерполяция (самый стабильный)

Если у тебя есть два ключевых кадра (например, рот закрыт → рот открыт), ИИ может дорисовать промежуточные кадры (In-betweening):

- Инструменты вроде **Live2D / Spine** с авто-сеткой или простые нейросети для интерполяции кадров (RIFE).
- Для пикселей хорошо работает метод морфинга в **Aseprite** (не ИИ, но делает промежуточные кадры за секунду).

### Вариант Б: Видео-ИИ → Пикселизация (Video-to-Video)

1. Берешь базовый спрайт Аластора.
2. Прогоняешь его через легкую видео-модель (например, **Kling, Runway Gen-3** или **Stable Video Diffusion**) с промптом: *«the character is talking / laughing / waving hand»*.
3. Получаешь короткое видео (MP4).
4. Нарезаешь его на кадры (через Python или видеоредактор).
5. Прогоняешь полученные кадры через даунскейл (уменьшение разрешения до размера твоих спрайтов) и квантование цветов (ограничение палитры).

### Вариант В: Генерация лип-синка (Lip-Sync)

Если нужно шевелить только ротом во время речи:

- Не генерируй всего персонажа заново.
- Сгенерируй или нарисуй всего **3–4 варианта нижней части лица**:
  1. Рот закрыт (улыбка).
  2. Рот слегка приоткрыт (виден зубастый оскал).
  3. Рот широко открыт («О» или крик).
- В коде просто подменяй слой рта поверх тела в зависимости от звука. Это выглядит в разы чище и аккуратнее, чем дергающаяся генерация от нейросетей.

## Практический вердикт: что лучше сделать прямо сейчас

1. **Не пытайся генерировать всю анимацию с нуля нейросетью:** ты потратишь часы на борьбу с несоответствием стилей, цветов и пропорций.
2. **Используй твои 47 картинок как основу:** это уже готовый, согласованный набор кадров.
3. **Используй ИИ точно:**
  - Если не хватает конкретного предмета (микрофон, чашка, зеленые радиоволны) — сгенерируй его через *Retro Diffusion* или любой генератор, уменьши до нужного разрешения и наложи как отдельный слой поверх Аластора.
  - Для генерации промежуточных кадров лучше использовать инструмент **Aseprite** или генератор спрайтов в **ComfyUI (IP-Adapter)**, скормив ему один из твоих 47 исходников в качестве референса стиля.